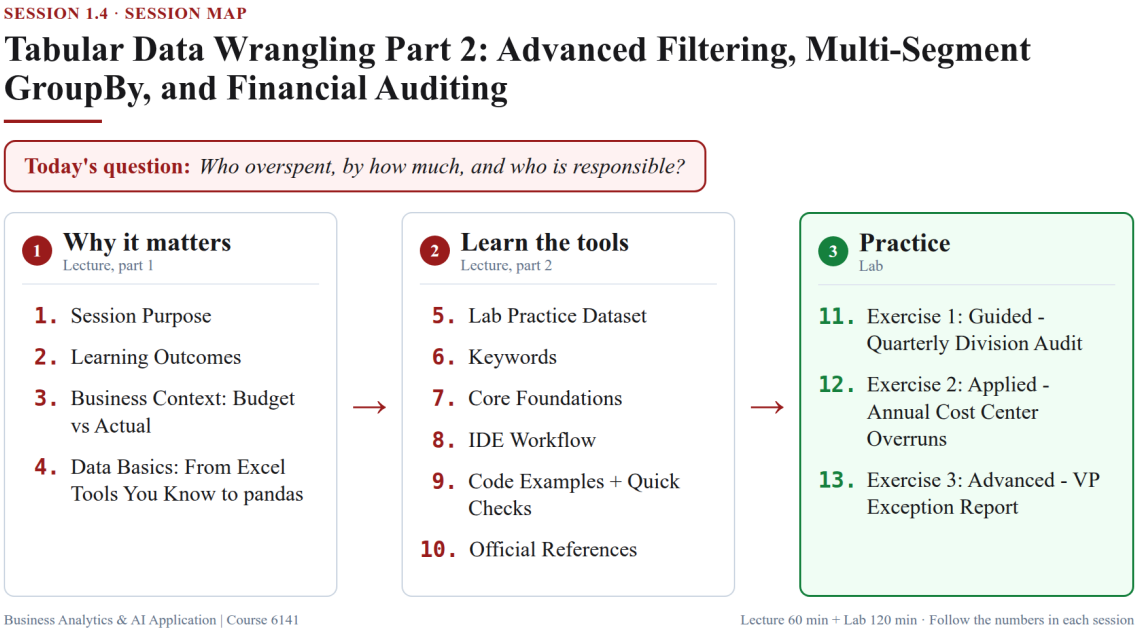
Session 1.4: Tabular Data Wrangling Part 2: Advanced Filtering, Multi-Segment GroupBy, and Financial Auditing

Course 6141: Business Analytics & AI Application | Reading for Week 4



Schedule: Week 4 | In-person classes (1:2 Delivery Ratio: 60 Min Lecture + 120 Min Lab)

1. Session Purpose

- Each division gets a spending budget every quarter, and finance compares it with what was actually spent
- Our case: 4 divisions x 4 quarters = 16 rows in one CSV file
- The finance director asks: who overspent, by how much, and who is responsible? Today, Python answers this for the whole year at once

2. Expected Learning Outcomes (By the end of this session, you will be able to)

1. "Which quarters went over budget?": filter rows with a Boolean mask (a `True`/`False` value for every row) and &

2. "How much did each division overspend this year?": summarize groups with `groupby()` (split rows into groups, like an Excel pivot table, which is a summary table with one row per group) and `.agg()`
3. "Who is responsible?": join tables with `pd.merge()` (join two tables on a shared column, like Excel's `VLOOKUP` formula, which looks up a value in another table) and fix a `KeyError`

3. Business Context: Budget vs Actual

- Four terms, one example (Consumer division, Q1):

Term	Meaning	Example
Budget	planned spending	`780,000`
Actual	real spending	`810,000`
Variance	Actual - Budget	`+30,000`
Variance %	Variance / Budget x 100	`+3.85%`

- Positive variance = overspent; negative variance = under budget.
- Cost center: the accounting code a division's spending is charged to, such as `CC-202` for Consumer. Each cost center has a responsible VP (Vice President).
- Company rule in our data: a quarter is flagged (`Audit_Status` = `Variance_Flag`) when the variance is more than 2%.
- Consumer division by quarter: Q1 `+3.85%` (flagged), Q2 `+1.92%` (over, within the rule), Q3 `+5.77%` (flagged), Q4 `-0.64%` (under budget).

4. Data Basics: From Excel Tools You Know to pandas

- Three Excel tools, in one line each:
  - **Filter:** hides the rows that do not match a rule, such as showing only the Consumer division.
  - **Pivot Table:** summarizes a large table by group. For example, put `Division_Name` in Rows and `Actual_Expense_USD` in Values to get total spending per division: 4 rows instead of 16.
  - **`VLOOKUP`:** a formula that looks up a value, such as a cost center code, in another table and returns a value from the same row, such as the VP's name. You tell it which column to return by number (for example, the 2nd column).
- The same three jobs in pandas:
  - Filter -> Boolean mask `df[mask]`: the rule is written in code, so it repeats exactly next quarter.
  - Pivot Table -> `df.groupby().agg()`: one line; run it again when new data arrives.
  - `VLOOKUP` -> `pd.merge()`: `VLOOKUP` returns a column by number; `pd.merge()` matches rows by column name, such as `Cost_Center_Code`.

5. Lab Practice Dataset

- File Link: `data/session_01_04_dataset.csv`
- Audited 16-row quarterly budget dataset (4 divisions x 4 quarters; each row is one division's spending in one quarter):
  - `Transaction_ID`: Unique record identifier (e.g., `TXN-5001`).
  - `Division_Name`: Operating business unit (Cloud, Consumer, Enterprise, Logistics).
  - `Cost_Center_Code`: Accounting cost center (CC-101, CC-202, CC-303, CC-404).
  - `Fiscal_Quarter`: Financial reporting quarter (Q1, Q2, Q3, Q4).
  - `Budget_Expense_USD`: Authorized spending budget in U.S. Dollars.
  - `Actual_Expense_USD`: Realized expenditure in U.S. Dollars.
  - `Audit_Status`: Internal audit classification (`Verified`, or `Variance_Flag` when spending is more than 2% over budget).

6. Keywords (Core Programming Concepts)

- `Boolean Mask`:
  - What: one `True` or `False` per row
  - Why: picks exactly the rows that meet a rule
- `& | ~`:
  - What: "and", "or", "not" for masks
  - Why: Python's `and` / `or` fail on whole columns
- `Split-Apply-Combine`:
  - What: split rows into groups, calculate each group, combine the results
  - Why: the idea behind every pivot table
- `groupby()` + `.agg()`:
  - What: `groupby()` splits; `.agg()` sums, counts, or averages each group
  - Why: yearly totals per division in one step
- `pd.merge()`:
  - What: joins two tables row by row
  - Why: adds facts from another table, such as the VP's name, with no copy-paste
- `Key Column`:
  - What: a column both tables share, such as `Cost_Center_Code`
  - Why: tells pandas which rows belong together

7. Core Programming & Computational Foundations

- `&`, not `and`: rules are checked row by row
  - Python's `and` expects one `True` or `False`; a mask holds 16 of them, one per row

  - `is_consumer and is_over` raises `ValueError: The truth value of a Series is ambiguous`
  - `is_consumer & is_over` gives one `True` / `False` per row; wrap each rule in `( )` when you write it inline

- Split-apply-combine: a pivot table written as code
  - Split: one group per `Division_Name`
  - Apply: `"sum"`, `"count"`, or `"mean"` in each group (for example, Consumer actual spending `810000.0 + 795000.0 + 825000.0 + 775000.0 = 3205000.0`)
  - Combine: one result row per group

- Merge vs `VLOOKUP`: match by name, not by position
  - `VLOOKUP` returns "the 2nd column"; insert a column into the lookup table and it returns the wrong one, which is easy to miss
  - `pd.merge()` uses column names, so moving columns does not matter

- `how="left"`: keep every row of the main table
  - Rows with no match stay, with an empty value (`NaN`, short for "Not a Number", pandas' mark for a missing value), so you can see which cost center has no VP
  - `how="inner"` would drop that row from the result

8. Development Workflow in Google Antigravity IDE

- `[1]` File Explorer: create your script with your name and the session code (for example, `Reacher_S14_demo_1.py`), and check the dataset is in `data/`.
- `[2]` Code Editor: write a few lines, run, repeat. Small steps keep errors small.
- `[3]` AI Agent Chat: on an error, capture the traceback (Python's error report) with `Win + Shift + S` (macOS: `Cmd + Shift + 4`), paste it with `Ctrl + V`, and ask why it failed.
- `[4]` Terminal: run `python` followed by your script name. Return code `0` means the script finished with no errors.

9. Hands-On Coding Micro-Examples

- Every example has three parts: **1. Input**, **2. Process**, and **3. Output**. Name each file with your own name and the session code, for example `Reacher_S14_demo_1.py` (pattern: `YourName_S14_demo_1.py`).

- *Micro-Example 1 (Find Overspending with a Boolean Mask):* `YourName_S14_demo_1.py`

```
# 1. Input: load the quarterly budget data
import pandas as pd
df = pd.read_csv("data/session_01_04_dataset.csv")

# 2. Process: Consumer quarters that spent more than budget
mask = (
    (df["Division_Name"] == "Consumer")
```

```
    & (df["Actual_Expense_USD"] > df["Budget_Expense_USD"])
)
overruns = df[mask]

# 3. Output: count and list them
print("Consumer overrun quarters:", len(overruns))
print(overruns[["Fiscal_Quarter", "Budget_Expense_USD",
                "Actual_Expense_USD"]])
```

  - `mask` holds one `True` / `False` per row. `&` means both rules must be `True`. `df[mask]` keeps only the `True` rows.
  - Result: 3 Consumer quarters (Q1, Q2, Q3) went over budget.

- *Quick Check 1 (~):*
  - New tool: `~` (not) turns every `True` into `False`, and every `False` into `True`. Why: find the opposite group, such as quarters that stayed within budget.
  - Task: in part **2. Process**, change `& (df["Actual_Expense_USD"] > df["Budget_Expense_USD"])` to `& ~(df["Actual_Expense_USD"] > df["Budget_Expense_USD"])`.
  - Check: the script prints `Consumer overrun quarters: 1`, and the table shows `Q4` (budget `780000.0`, actual `775000.0`). The label still says "overrun", since only the rule changed.

- *Micro-Example 2 (Total Each Division with `groupby()`):* `YourName_S14_demo_2.py`

```
# 1. Input: load the quarterly budget data
import pandas as pd
df = pd.read_csv("data/session_01_04_dataset.csv")

# 2. Process: yearly totals for each division
div_summary = df.groupby("Division_Name").agg(
    Total_Budget_USD=("Budget_Expense_USD", "sum"),
    Total_Actual_USD=("Actual_Expense_USD", "sum"),
    Quarters=("Transaction_ID", "count"),
)
div_summary["Variance_USD"] = (
    div_summary["Total_Actual_USD"]
    - div_summary["Total_Budget_USD"]
)

# 3. Output: one row per division
print(div_summary[["Total_Actual_USD", "Variance_USD"]])
```

  - `groupby("Division_Name")` makes 4 groups. Inside `.agg()`, each item reads `New_Name=("Column", "sum")`: the new column name, the source column, and the calculation.
  - Result: 4 rows instead of 16, one per division.

- *Quick Check 2 (`.sort_values()`):*
  - New tool: `.sort_values()` reorders rows by one column; `ascending=False` puts the largest first. Why: the director reads the biggest overspender first.

  - Task: add one line at the end of part **2. Process**: `div_summary = div_summary._____("Variance_USD", ascending=False)`
  - Check: the first row is `Consumer` (`85000.0`), then `Logistics` (`28000.0`).

- *Micro-Example 3 (Add VP Names with `pd.merge()`):* `YourName_S14_demo_3.py` (VP names are fictional)

```
# 1. Input: budget data and a VP list (fictional names)
import pandas as pd
df = pd.read_csv("data/session_01_04_dataset.csv")
vp_info = pd.DataFrame({
    "Cost_Center_Code": ["CC-101", "CC-202", "CC-303", "CC-404"],
    "VP_Lead": ["Sarah Chen", "Marcus Vance",
                "Elena Rostova", "David Kim"],
})

# 2. Process: add VP names, then flag spending over budget
ledger = pd.merge(df, vp_info, on="Cost_Center_Code", how="left")
ledger["Over_Budget"] = (
    ledger["Actual_Expense_USD"] > ledger["Budget_Expense_USD"]
)

# 3. Output: show the first three rows
print(ledger[["Transaction_ID", "VP_Lead", "Over_Budget"]].head(3))
```

  - `pd.DataFrame({...})` builds a small table in code: each column name, then its list of values.
  - `on=` names the key column; `how="left"` keeps all 16 rows. `>` makes a `True` / `False` column.

- *Quick Check 3 (`.value_counts()`):*

  - New tool: `.value_counts()` counts how often each value appears in a column. Why: answers "how many quarters were over budget?" in one line.
  - Task: add one line to part **3. Output**: `print(ledger["Over_Budget"]._____())`
  - Check: the Terminal shows `False  8` and `True  8`: half of the 16 quarters went over budget.

10. Official Python Documentation & References

- Pandas User Guide: GroupBy: Split-Apply-Combine.
- Pandas User Guide: Merge, Join, Concatenate and Compare.
- Python PEP 8: Guidelines for Line Breaks around Binary Operators.

11. Exercise 1: Guided Practice - Quarterly Division Audit

- *Step 1: Open Script:* In Google Antigravity IDE, create a new script named `YourName_S14_audit.py`, replacing `YourName` with your own name (for example, `Reacher_S14_audit.py`).
- *Step 2: Group by Division and Quarter:* Type the following code:

```
# 1. Input: load the quarterly budget data
import pandas as pd
df = pd.read_csv("data/session_01_04_dataset.csv")

# 2. Process: variance per row, then group by two columns
df["Variance_USD"] = df["Actual_Expense_USD"] - df["Budget_Expense_USD"]
df["Variance_Pct"] = df["Variance_USD"] / df["Budget_Expense_USD"] * 100.0
quarterly_audit = df.groupby(["Division_Name", "Fiscal_Quarter"]).agg(
    Total_Actual=("Actual_Expense_USD", "sum"),
    Total_Variance=("Variance_USD", "sum"),
    Mean_Variance_Pct=("Variance_Pct", "mean"),
)

# 3. Output: one row per division and quarter
print("Quarterly Division Audit Summary:\n", quarterly_audit)
```

  - Grouping by a list of two columns gives one row per division and quarter: 16 rows, labeled by both values.

- *Step 3: Verify Output Integrity:* Confirm that quarters with overruns show positive `Total_Variance` values. 8 of the 16 rows should be positive.
- *Step 4: Execute in Terminal:* Run the script via Panel `[4]` (Terminal) using `python YourName_S14_audit.py` and confirm return code `0`.

12. Exercise 2: Applied Practice - Annual Cost Center Overruns

- *Coding Challenge:* In `YourName_S14_practice.py`, find the cost centers that overspent for the whole year.
- *Task 1 (Annual Cost Center Aggregation):* Group the data by `Cost_Center_Code`, sum budget and actual spending, compute the yearly variance %, and keep cost centers more than `1.0`% over budget.
- *Task 2 (Deliberate KeyError & AI Diagnosis):* Merge the result with `vp_info` (from Micro-Example 3), but type `on="Cost_Center"` on purpose. Read the `KeyError` (the column name does not exist), capture the traceback with `Win + Shift + S`, paste it into Panel `[3]` (AI Agent Chat) with `Ctrl + V`, and review the AI's explanation of join keys.
- *Task 3 (Audit Brief Output):* Fix the key to `on="Cost_Center_Code"`, then print the cost center, the VP, and the overrun in USD.
- *Success Check:* 2 cost centers: `CC-202` (Marcus Vance, `85000.0`, 2.72%) and `CC-404` (David Kim, `28000.0`, 1.15%); the script ends with return code `0`.

13. Exercise 3: Advanced Practice - VP Exception Report

- *Goal:* Build an exception report that tells the finance director which VPs must explain their spending. A VP is escalated when the cost center has repeated flagged quarters **and** overspent for the whole year.
- *What you need:*

  - File: `data/session_01_04_dataset.csv` (16 rows, described in **5. Lab Practice Dataset**).

  - The `vp_info` table from Micro-Example 3 (VP names are fictional).
  - Script name: `YourName_S14_challenge.py`.
  - Tools you already know: Boolean masks, `groupby()` + `.agg()`, `pd.merge()`, `.sort_values()`, and `.to_csv()`.

- *Requirements (follow them in order):*

  1. Add `Variance_USD` = `Actual_Expense_USD - Budget_Expense_USD` and `Variance_Pct` = `Variance_USD` / `Budget_Expense_USD` x 100, rounded to 2 decimal places.
  2. Add `My_Flag` = `True` when `Variance_Pct > 2.0`. Check your rule: count the rows where `My_Flag` disagrees with `Audit_Status == "Variance_Flag"`. The count must be `0`.
  3. **Flagged summary:** keep only the flagged rows, then group by `Cost_Center_Code` and compute `Flagged_Quarters` (count of rows), `Flagged_Overrun_USD` (sum of `Variance_USD`), and `Worst_Variance_Pct` (largest `Variance_Pct`).
  4. **Yearly summary:** using all 16 rows, group by `Cost_Center_Code`, sum budget and actual spending, and compute `Year_Variance_Pct` = (actual - budget) / budget x 100, rounded to 2 decimal places.
  5. Start from `vp_info` and merge in the yearly summary, then the flagged summary, both with `on="Cost_Center_Code"` and `how="left"`. If a cost center had no flagged quarter, its empty values must become `0`.
  6. Add `Escalate` = `True` when `Flagged_Quarters >= 2` **and** `Year_Variance_Pct > 1.0`.
  7. Sort by `Flagged_Overrun_USD`, then by `Worst_Variance_Pct`, both largest first. Print the table and save it to `data/vp_exception_report.csv` without row labels.

- *New tools for this exercise:*

  - `.reset_index()`: after `groupby()`, the group names (here, the cost center codes) become the row labels. `.reset_index()` turns them back into a normal column, so you can merge on `Cost_Center_Code`.
  - `.fillna(0)`: replaces empty values (`NaN`) with `0`, for example `report["Flagged_Quarters"] = report["Flagged_Quarters"].fillna(0)`.
  - Sorting by two columns: `.sort_values(["Flagged_Overrun_USD", "Worst_Variance_Pct"], ascending=False)`. The second column only decides ties in the first.
  - `"max"` inside `.agg()` returns the largest value in each group, just as `"sum"` returns the total.

- *Hints:*

  - Build the two summaries as two separate tables first, print each one, and only then merge.
  - `(df["My_Flag"] != (df["Audit_Status"] == "Variance_Flag")).sum()` counts disagreements, because each `True` counts as 1.

- *Self-Check (expected results):*

  - Rule check: `0` disagreements; 6 of the 16 quarters are flagged.
  - Table order: `CC-202` Marcus Vance (2 flagged, `75000.0`, worst `5.77`), `CC-404` David Kim (2, `45000.0`, `4.92`), `CC-303` Elena Rostova (1, `10000.0`, `3.12`), `CC-101` Sarah Chen (1, `10000.0`, `2.22`).
  - `Escalate` is `True` only for Marcus Vance (`Year_Variance_Pct` `2.72`) and David Kim (`1.15`). Elena Rostova and Sarah Chen each had one flagged quarter but finished the year at or under budget.